Programlama Laboratuvarı I Proje III
Yazarlar ve İşbirliği Analizi Graf Yapısı

Farahnozkhon Dilovarovna Masumova
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
220201142

Nazire İpek Agan
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
230201024

Özet– Bu projede, .xlsx formatındaki akademik veri setini ayrıştırıp sınıflandırarak manuel bir graf yapısı oluşturulmuştur. Yazarlar grafın düğümlerini, iş birlikleri kenarlarını ve iki yazarın ortak makale sayıları da kenar ağırlıklarını oluşturmuştur. Aynı zamanda bu graf yapısı görselleştirilmiş ve graf üzerinde seçilen bir işlemin uygulanması için butonlu bir görsel arayüz tasarlanmıştır. Veri saklamak için birçok alanda kullanılan graf yapısını manuel fonksiyonlarla oluşturarak ve bu yapı üzerinde arama ve sıralama işlemleri yaparak veri saklama ve analizi üzerinde önemli konseptler uygulanmıştır.

I. GİRİŞ

Graf veri yapısı, veri bağlantıları temsil etmek için oluşturulan; düğümler, bu düğümlerin bağlantılarını gösteren kenarlar ve ağırlıklardan oluşan ve bilgisayar bilimlerinde çok sık kullanılan temel veri yapılarından biridir. Kenarların yönü olup olmamasına göre graflar yönlü ya da yönsüz graf olarak, kenarların ağırlıkları olup olmamasına göre de ağırlıklı ve ağırlıksız graflar olarak adlandırmak mümkündür. Graf veri yapısı, internet ağı, arama motorları, sosyal medya platformları, haritalandırma sistemleri, işletim sistemleri gibi bilgisayar bilimlerinin birçok alanında olduğu gibi, günlük hayatta da birçok veri ilişkisini modellemek için kullanılır.

Python dilinde yazılan bu projede ise akademik bir veri setinden ayrıştırılan yazar ilişkilerine ve ilişkilerinin tekrarına göre manuel olarak bir veri yapısı oluşturulmuştur. Yazarların düğümler, ortak makale sayılarının ise kenar ağırlıkları olarak temsil edildiği bu yapıda oluşturan graf görselleştirilmiş ve kullanıcıya sunulan uygulamanın orta segmentine yerleştirilmiştir. Bu graf temsiline eklenen uzaklaştırma yakınlaştırma özelliği ve üzerine tıklanan düğümlerde yazar adları ve makale listesine erişilme özelliği gibi işlevler eklenerek kullanıcıya kolay bir interaktif kullanım sağlanmıştır. Uygulamanın sağ tarafında farklı işlemler yapan yedi adet buton, sol tarafında da bu butonlara tıklanarak yapılan işlemlerin çıktısını göstermek için bir ekran yerleştirilmiştir. Kullanıcının uygulamadaki butonlara basarak graf üzerinde çeşitli işlemler yaptığı bu graf uygulamasında, birinci butonda kullanıcının girdiği iki yazarın arasında kenar ağırlıklarına göre en kısa yol bulunmuş ve bir kuyruk olarak saklanmış, ikinci butonda bir yine kullanıcının girdiği bir yazarın tüm bağlantıları düğüm ağırlıklarına göre sıralanmış, üçüncü butonda birinci butonda girilen yazarların en kısa yolundan oluşan kuyruktaki verilerden bir Binary Search Tree oluşturulmuş, dördüncü butonda kullanıcının girdiği yazarın ve iş birliği yaptığı yazarların tüm düğümlere olan en kısa uzaklığı hesaplanmış, beşinci butonda kullanıcının girdiği yazarın toplam iş birliği sayısı gösterilmiş, altıncı butonda graf yapısında en çok iş birliği yapan yazar gösterilmiş, yedinci adımda ise kullanıcının girdiği yazardan başlayarak gidilebilecek düğüm sayısına göre en uzun yol hesaplanmış ve gösterilmiştir. Bu çıktılar orta segmentteki graf yapısı üzerinde gösterilmiştir.

II. SINIFLARIN OLUŞTURULMASI

Yazar, Makale, Graf adlı üç sınıf ile, graf yapısını oluşturmak için gerekli olan bilgiler akademik veri setini içeren .xlsx uzantılı dosyadan ayrıştırılıp saklanmıştır.

Yazar sınıfı akademik veri setindeki yazarları yani graf yapısında düğümleri temsil eder ve bu sınıfın her bir nesnesi veri setindeki bir yazara karşılık gelmektedir. *orcid* değişkeni her bir yazarın ORC-ID (Open Researcher and Contributor ID) kimliğine, *name* değişkeni her bir yazarın adına tekabül eder. Sınıfın yapıcı metodu ise *orcid* ve *name* değerlerini alarak nesneleri oluşturmaktadır. *papers* listesi yazarın yayınladığı makaleleri tutarken *connections* listesi de yazarın iş birliği yaptığı yazarları saklayan bir listedir.

Makale sınıfı akademik veri setindeki makaleleri temsil eder ve bu sınıfın her nesnesi veri setindeki makalelere karşılık gelmektedir. *doi* değişkeni makalenin DOI (Digital Object Identifier) kimliğini, *title* değişkeni de makalenin adını tutmaktadır. *authors* listesi ise makaleyi yazan yazar nesnelerini saklar. Sınıfın yapıcı metodu *doi* ve *title* değişkenlerini alarak nesneleri oluşturur.

Graf sınıfı, verilerin bağlantılarına göre oluşturulan graf yapısını temsil etmektedir. *nodes* listesi düğümleri yani yazarları temsil eder. *dugum_ekle*(self, yazar) metodu grafa düğüm yani *Yazar* sınıfının nesneleri olan yazarları ekler. İlgili yazar grafın düğümleri içinde yoksa listeye eklenir. *Kenar_ekle(self, yazar1, yazar2, ortak_makale_sayısı)* metodu ise graf yapısına kenar eklemek için kullanılır. *yazar1* ve *yazar2 nesnelerinin* birbirlerinin *connecitons* listesinde olup olmadığı kontrol edilir. Örneğin *yazar2* nesnesi *yazar1*'nin *connections* listesinde yoksa, *yazar2* ve *ortak_makale_sayısı* bir tuple olarak *yazar1*'in *connections* listesine eklenmektedir. Aynı işlem *yazar2*'nin *connections* listesi için de yapılır.

tum_yollar_bul(self, yazar_a, yazar_b, path = [], toplam_agirlik = 0) fonksiyonu başlangıç düğüm *yazar_a* ile hedef düğüm *yazar_b* arasındaki tüm yolları bulmaktadır. *yazar_a* grafın düğümleri içerisinde değilse boş bir liste döndürülür. *yol,* tek bir yolda ziyaret edilen düğümlerin listesidir. Tüm olası *yol*'ları bulmak için *yazar_a*'nın *connections* listesindeki her komşu ve onların ağırlığı kontrol edilir. Eğer komşu düğüm daha önce ziyaret edilmemişse *tum_yollar_bul* fonksiyonu recursive olarak tekrar çağırılır. Her ağırlık değeri *toplam_agirlik* değişkenine eklenerek yolun toplam ağırlığı tutulur. *yazar_b* düğümüne ulaşıldıysa hedefe ulaşılmış demektir. Recursive fonksiyondan dönen tüm yollar ağırlıklarıyla beraber tuple yağıları *tum_yollar* listesine eklenir. Ve fonksiyon sonuç olarak tüm bu yolların saklandığı *tum_yollar* listesini döndürür.

en_kisa_yol_bul(self, tum_yollar) fonksiyonu, *tum_yollar_bul* fonksiyonundan döndürülen iki düğüm arasında gidilebilecek tüm yolları içeren listes içinde ağırlık değeri en küçük olanı, yani en kısa yolu döndürür. Parametre olarak alınan *tum_yollar* listesi içinde bulunan yol listesi ve ağırlık olmak üzere iki değişken tutan tuple'lar listesi boş değil mi diye kontrol edilir. Eğer liste boş değilse yani yollar mevcutsa *min()* fonksiyonu kullanılarak tuple yapılarının ikinci ögeleri olan yol ağırlıkları kıyaslanır ve içlerinden en kısa olan döndürülür.

en_kisa_yollar_hesapla(self, yazar) fonksiyonu, bir yazardan olası tüm yazarlara en kısa yolları hesaplar. Parametre olarak başlangıç düğümü olan *yazar* nesnesini alır. Fonksiyon içinde yazar değişkeninin *tum_yollar_bul* fonksyionu ile *diger_yazar* değişkenine giden tüm yolları bulunur ve bunlar arasından da *en_kisa_yol_bul* metodu ile en kısa yol bulunur. Bu işlem tüm *diger_yazar* değişkenleri için tekrarlanır ve bulunan her liste, *diger_yazar'ın orcid* değişkeni ve yolun ağırlığı ile beraber *en_kısa_yollar* sözlüğüne kaydedilir ve en sonda bu sözlük fonksiyondan döndürülür.

kuyruktan_bst_olustur(self, queue) fonksiyonu, *queue* listesindeki düğümleri kullanarak bir Binary Search Tree oluşturan fonksiyondur. Binary Search Tree (İkili Arama Ağacı), sağ taraftaki değerlerin kökten küçük, sol taraftaki değerlerin kökten büyük olduğu bir arama ağacıdır. Bu *queue* listesi zaten iki düğüm arasındaki en kısa yoldur ve bu yoldaki düğümleri içeren liste *en_kisa_yol_bul* fonksiyonundan dönen listedir. TreeNode sınıfı, oluşturulacak olan ağacın düğümlerini temsil eder ve düğümün kendisi *author*, sağı *right* ve solu *left* değişkenleri ile tutulur. Fonskiyon içinde tanımlı olan *ekle(root, author)* fonksiyonu ise BST'ye yeni bir düğüm eklemek için kullanılır. Eğer mevcut kök boşsa, *TreeNode* sınıfından yeni bir düğüm oluşturur

ve döndürür. Eğer eklenmek istenen yazarın X değeri kök yazarın yazdığı makale sayısı küçükse onu ağacın soluna ekler, kök yazarın yazdığı makale sayısından büyük ise sağ tarafa ekler. *kuyruktan_bst_olustur* fonksiyonu, gelen *queue* değişkeni içindeki her yazar üzerinde döngüyle gezer ve *ekle* fonksiyonu ile yazarlar ağaca eklenir. Tüm yazarlar ağaca eklendikten sonra ağacı kökü fonksiyondan return edilir.

ortak_yazar_sayisi_hesapla(self, yazar) fonksiyonu ise bir yazarın kaç farklı yazar ile ilişkisi olduğunu hesaplar. Bunun için yazarın yani *Yazar* sınıfından *yazar* nesnesinin *connections* listesini uzunluğu *len()* fonksiyou ile bularak bu değeri döndürür ve yazarın kaç yazarla iş birliği yaptığına ulaşılır.

en_cok_isbirligi_yapan_yazari_bul(self) fonksiyonu graftaki tüm düğümler yani veri setindeki yazarlar içinde en çok iş birliği yapan yazarı bulur ve bu yazarla birlikte onun iş birliği sayısını döndürür. *en_cok_isbirligi_yapan_author* değişkenine tüm yazarlar içinden iş birliği sayısı en fazla olan atanır. Aynı şekilde *toplam_isbirligi* değişkenine de bu yazarın toplam iş birliği sayısı atanır ve bu iki değer fonksiyondan döndürülür.

en_uzun_yol_bul(self, yazar_id) fonksiyonu ise gönderilen ID'li yazardan başlayarak graf üzerinde gidilebilecek en uzun yolu bulur. Öncelikle gönderilen ID'ye sahip yazarın hangi yazar olduğunu bulmak için grafın düğümlerini gezerek aynı ID'ye sahip yazarı bulur ve bunu *author* değişkenine atar. Eğer böyle bir yazar yoksa hata mesajı yazdırılır ve boş liste döndürülür. e*n_uzun_yol* adlı, en uzun yolu saklamak için bir liste, *ziyaret_edildi* ziyaret edilen tüm yazarları tutan bir küme ve *yigin* yazarları ve yollarını tutan bir listedir. İlk yazara *yigin* listesine eklendikten sonra bir döngü başlatılır ve bu liste boşalana kadar döngü devam eder. Döngünün her bir adımında *yigin* listesinden bir yazar (*simdiki_author)* ve yazara giden yol(*path*) alınır. Yazar şimdiye kadar ziyaret edilenler listesine (*ziyaret_edildi*) eklenir. Sonra bulunan en uzun yol ile şimdiki yol karşılaştırılır ve en uzun yoldan uzunsa *en_uzun_yol* bu yolla değiştirilir. Daha sonra, mevcut yazarın *connections* listesi içinde gezilir, komşu yazar listeye eklenmemiş ve daha önce ziyaret edilmemişse o komşu yazara giden yol *yigin* listesine eklenir. Her adımda bu işlemler tekrar eder ve döngü tamamlandığında elde edilen *en_uzun_yol* listesi fonksiyon tarafından return edilir.

Bu sınıf yapıları ve fonksiyonlar sayesinde dosyadan elde edilen veriler Yazar ve Makale sınıflarının nesnesi olarak saklanırken Yazar nesnelerinin bağlantılarıyla manuel olarak oluşturulan graf yapısı Graf sınıfından türetilen bir nesne olmuştur.

III. VERİ SETİNİN AYRIŞTIRILMASI VE İLGİLİ NESNELERİN OLUŞTURULMASI

Kod dizininde bulunan PROLAB 3 - GÜNCEL DATASET.xslx dosyasını okumak için veri analizi kütüphanesi Pandas kullanılmıştır. Kütüphaneyi indirmek için *pip install pandas* komutu kullanılmış ve *import pandas as pd* ile koda dahil edilmiştir.

Akademik veri setinde *orcid* sütununda yazarın id'si, *doi* sütununda makalenin id'si, *author_position* sütununda yazarın makaleyi yazan yazarların arasında kaçıncı sırada olduğu bilgisi, *author_name* sütununda orcid'i verilen yazarın adı, *coauthors* sütununda makaleye katkıda bulunmuş tüm yazarların adları, *paper_title* sütununda ise doi'si verilmiş olan makalenin adı tutulmaktadır. *dosya* değişkenine dosyanın adı kaydedilmiş ve bu dosya değişkeni Pandas kütüphanesinin *read_excel* fonksiyonu ile okunarak *veriseti* isimli *DataFrame* nesnesine kaydedilmiştir. *DataFrame*, Pandas kütüphanesinin yapılarından biridir ve satırlar ve sütunlar olarak düzenlenmiş veriyi temsil eder.

Daha sonra *yazarlar* ve *makaleler* adında iki farklı dictionary tanımlanmıştır. *yazarlar* sözlüğü ORCID anahtarı ve yazar ismi değerlerini tutmak, *makaleler* sözlüğü de DOI anahtarı ve makale ismi değerlerini tutmak için kullanılmıştır. Aynı şekilde *yazar_ismi_haritası* isimli bir default(list) sözlüğü tanımlanmış ve yazar isimlerine göre yazarları tutmak için kullanılmıştır. Excel dosyasından okunan verilerin okumasına başlamadan önce *Graf* sınıfından *graf* adlı bir nesne oluşturulmuştur.

Öncelikle verideki her satırı gezerek yazar bilgileri oluşturulmuştur. Bunun ilk adımı da *orcid* sütununda verilen tüm ORCID numaraları ve *author_name*

sütununda verilen isimleri birleştirerek yazar nesnelerini oluşturmaktır. *orcid* satırdaki yazarın ORCID numarasını, *author_name* satırdaki yazarın adını tutar. Bu ORCID değeri, yazarlar sözlüğünde yoksa yeni bir *Yazar* nesnesi oluşturulur ve *yazarlar* sözlüğüne eklenir. Aynı zamanda *yazar_ismi_haritasi* sözlüğüne adı anahtar olacak şekilde bu yazar eklenir. Eğer ORCID değeri *yazarlar* sözlüğünde mevcutsa ve adı *author_name* sütunundan daha önce kaydedilen ada eşit değilse, nesne tekrar oluşturulur ama listede bu *orcid* numarasıyla ilk karşılaşılan *author_name* ile oluşturulduğundan aynı obje elde edilir. Bu sayede listede birden çok kez karşılaşın ORCID numaraları, adlarının ne olduğundan bağımsız olarak tek bir *Yazar* nesnesi olarak *yazarlar* sözlüğüne kaydedilirler.

Veri tekrar satır satır işlenmek üzere döngü başlatılır. *coauthors* sütunundaki verileri işlemek için, öncelikle bu satırın her hücresindeki *coauthors* listeleri *ast.literal.eval* ile liste formatındaki yazarlar gerçek bir Python listesine dönüştürülür. Satırı işlemek için sırasıyla şu işlemler gerçekleştirilir: yazarlar sözlüğünden satırdaki yazar alınır ve *merkez_yazar* olarak saklanır. Eğer makaleler listesine *doi* daha önce eklenmemişse o *doi* ve *paper_title* ile yeni bir makale nesnesi oluşturulur ve *merkez_yazar* bu makale üzerinde çalıştığı doğrulandıktan sonra makale yazarın *papers* listesine eklenir, aynı şekilde yazar da makalenin *authors* listesine eklenir. Daha sonra *author_position* sütunundaki değerin 1 eksiği indeksteki yazar *coauthors* sütunundaki listeden çıkarılır ve tekrar işlenmez. Çünkü bu yazar *author_name* sütunundaki yazar olduğundan ve bir önceki adımda makalenin *authors* listesine eklendiğinden buna gerek kalmamıştır.

coauthors sütunundaki listeyi işlemek için yazarlar isimleri üzerinden aranırlar. Eğer listedeki yazarın ismi, *yazarlar_isim_haritası* sözlüğünde bulunursa ve yazarın ORCID değeri satırdaki ORCID değerine eşitse bu *found_author* olarak atanır. Eğer *found_author* hâlâ bulunamadıysa, *yazar_ismi_haritası* içinde mevcut olup olmadığına bakılır. Sözlük içinde varsa aynı isimli yazar olan seçilir, yoksa yazara yeni bir ORCID değeri verilir ve yeni bir *Yazar* nesnesi oluşturulur. Bu yeni nesne hem yazarlar sözlüğüne hem de *yazar_ismi_haritası* sözlüğüne eklenir.

Yeni yazarlar ve makaleler arasında ilişki kurmak için ise *found_author* yazarının *papers* listesine satırdaki makale eklenir ve makalenin *authors* listesine de yeni yazar eklenir. Bu eklemeler bu nesnelerin önceden listelerde olup olmadığı kontrol edildikten sonra yapılır.

IV. GRAF YAPISININ OLUŞTURULMASI

Manuel olarak oluşturulan graf yapısında yazarlar düğümlere, kenarlar ise yazarlar arasındaki ilişkilere karşılık gelir. Öncelikle yine veri setini satır satır taramak için bir döngü başlatılır. Yine *orcid, author_name, doi, paper_title ve author_position* değişkenleri aynı isimli sütunlarda karşılık gelen verileri tutar. Önceki adımda olduğu gibi *ast.literal_eval* kullanılarak *coauthors* sütunundaki liste bir Python listesine dönüştürülür. *orcid* sütunundaki yazar yazarlar sözlüğünden alınır ve *merkez_yazar* değişkenine atanır. Satırdaki *coauthors* üzerinde bir döngü başlatılır ve *yazar_ismi_haritası* sözlüğünde ilk eşleşen yazar *coauthor* olarak atanır. *merkez_yazar* ve *coauthor* nesnelerinin farklı olup olmadığı kontrol edilir ve bu sayede aynı yazarın ilişki kurması engellenmiş olur. Daha sonra bu iki yazarın da *papers* listelerindeki *doi* değerleri *set()* ile bir kümeye dönüştürülür ve bu kümelerin ortak elemanları *intersection()* metodu ile bulduktan sonra, *len()* ile bu kesişimin uzunluğunun hesaplanır. Bu değer, *ortak_makale_sayısı* içinde tutulur ve graf yapısında bir kenarın ağırlığına tekabül eder.

Eğer *ortak_makale_sayısı* 0 değerinden büyükse, bu iki yazar arasında ilişki var demektir ve Graf sınıfının *kenar_ekle* metoduna bu iki yazar nesnesi ile beraber *ortak_makale_sayısı* değeri parametre olarak gönderilir ve kenar oluşturulmuş olur. Daha sonra, *yazarlar* sözlüğündeki tüm yazar nesneleri Graf sınıfının *dugum_ekle* metodu ile graf yapısına düğüm olarak eklenir.

Bu işlemler sonucunda manuel olarak oluşturulmuş, her düğümü eşsiz ORCID'li bir yazarı temsil eden ve

her kenarı bu yazarların ortak makale sayısını ağırlık olarak alan bir graf veri yapısı elde edilmiştir.

V. GÖRSELLEŞTİRME

Projede görselleştirme için bir Python kütüphanesi olan PyQt5 kullanılmıştır. PyQt5, çok yaygın kullanılan Qt platformunun özelliklerini Python dilinde kullanmayı sağlayan, interaktif uygulamalardan görsel oluşturmaya kadar birçok amaç için kullanılan grafiksel arayüz kullanıcı kütüphanesidir. Bu projede de *pip install PyQt5* kullanılarak kütüphane indirilmiş ve QtWidgets, QtGui ve QtNetwork gibi PyQt5 modüllerinden yararlanılmıştır.

VI. İNTERAKTİF UYGULAMA

Projenin interaktif uygulaması üç bölümden oluşur: sağda butonlar, ortada manuel olarak oluşturulmuş, düğümlerine tıklanabilen graf görseli ve sol tarafta ise butonlara tıklanarak yapılan işlemlerin sonucunun gösterildiği bir çıktı ekranı. *TiklanabilirElips(QGraphicsEllipseltem)* sınıfı, interaktif bir elips şekli tanımlar. Şekle tıklandığında yazara ait bilgiler, ORCID değeri ve yazdığı makaleler bir diyalog kutusu içinde gösterilmektedir. Bu sınıf, PyQt5'in özel sınıfı olan *QGraphicsEllipseltem* sınıfından üretilmiştir ve bir ellips nesnesi oluşturarak onu bir Yazar nesnesiyle ilişkilendirir. Parametre olarak elipsin konumunun x ve y koordinatları, radius olarak elipsin yarıçapı, author olarak da elipsin temsil ettiği *Yazar* nesnesini alır.

mousePressEvent metodu, kullanıcı graf düğümünü tıkladığında çalıştırılır. Tıklanan yazarın ORCID değeri ve adı konsola yazdırılır. Bu fonksiyon içerisindeki *QDialog* ile tıklanan yazarın adını göstermek için pencere oluşturulur, *QScrollArea* ile diyalog kutusu içinde gösterilecek metnin uzun olması halinde kaydırma özelliği eklenir, tıklanan yazara ait makale listesi self.author.papers listesinden alınır ve makalelerin adını yazdıran bir metin listesi oluşturulur. Yazarın ORCID'i ve makalelerini içeren *message, QLabel* aracılığıyla diyalog penceresine eklenir.

VII. GRAF GÖRSELİ

GrafArayuzu(QMainWindow) sınıfı, bir PyQt5 kütüphanesinin *QMainWindow* sınıfından türetilmiştir ve düğümleri, düğüm ilişkilerini kısacası graf yapısını görselleştirmek için kullanılmıştır. Sol, sağ ve orta segmentlere tekabül eden l*eft_layout, right_layourt, center_layout* ile bir görsel arayüz oluşturulmuştur. *QGraphicsScene* ve *QGraphicsView* ile grafın görselleştirilmesi için bir alan oluşturulur. Graf görseli üzerinde yazarın pozisyonlarını takip etmek için başlangıçta bir *node_positions* sözlüğü tanımlanır. 1. İSTER, 2. İSTER gibi buton başlıklarını tutmak için *button_labels* sözlüğü tanımlanır. Daha sonra *grafi_ciz* fonksiyonu çağrılır.

grafi_ciz() fonksiyonunda, graf düğümleri içindeki tüm yazarların toplam makale sayısı bulunur ve *toplam_makale_sayisi* değişkeninde tutulur. Aynı şekilde toplam düğüm sayısı ve *max_makale_sayisi* değişkeniyle en çok makale yazan yazarın makale sayısı bulunur. Yazarların ortalama makale sayısı da bulunarak bu değer *ortalama_makale_sayisi* içinde tutulur. Her bir yazar için *TiklanabilirElips* nesnesi oluşturulduktan sonra ve düğümün boyutu ortalama makale sayısına göre belirlenir. Ortalamanın yüzde yirmi ve üzerinde makale yazmış yazarlar daha büyük boyutta ve daha açık renkle gösterilirken, ortalamanın yüzde yirmi altında makale yazmış yazarlar daha küçük boyutta ve daha koyu renkle gösterilirler.

Düğümler arasındaki kenarları çizmek için, yazarların birbirlerinin *connections* listesinde olup olmadığı araştırılır ve bağlantıları varsa düğümün merkez koordinatından başlayarak diğer düğümün merkez koordinatına bir çizgi çizilir. Çizilen kenarlar *drawn_edges* listesine kaydedilir.

yazar_bilgisi_goster fonksiyonu bir düğüm üzerine tıklandığında kullanıcıya yazar bilgilerini gösterir. *QMessageBox* ile bu yazar bilgisi kullanıcıya mesaj kutusu içinde gösterilir.

wheel_event fonksiyonu mouse ile graf görselinin zoom-in ve zoom-out işleminin gerçekleşmesini sağlar. Tekerleği yukarı kaldırmak yakınlaştırır, aşağı kaydırmak uzaklaştırır.

buton_tıklandı fonksiyonu ise her bir butona özel olarak atfedilmiş görevleri tanımlayan fonksiyondur.

VIII. İSTERLER

1. İSTER: kullanıcıdan girilen iki ID'ye göre iki yazarın arasındaki kenar ağırlıklarına göre en kısa

yol bulunur ve bu yol graf üzerinde gösterilir. Bunun için *buton_tıklandı* fonksiyonundaki kontrol yapısında buton metni 1. İSTER olan butona tıklanılması gerekmektedir. *QInputDialog.getText* kullanılarak kullanıcıdan iki yazar ORCID değeri alınır. Graf düğümleri arasında bu kimliklere sahip yazarlar aranır ve bulunamazlarsa hata mesajı verir. Eğer bu iki yazar graf düğümleri içinde mevcutsa daha önce "Sınıfların Oluşturulması" bölümünde işleyişi açıklanan Graf sınıfının bir fonksiyonu olan *tum_yollar_bul(yazar1, yazar2)* fonksiyonu ile bu iki yazar arasındaki tüm yollar bulunur ve *tum_yollar* değişkeninde saklanır. Soldaki çıktı ekranına tüm yollar sıralanır ve her yolun yanında geçilen kenar ağırlıklarının toplamı yazılır. Yine daha önce "Sınıfların Oluşturulması" bölümünde işleyişi açıklanan *en_kisa_yol_bul* fonksiyonuna parametre olarak gönderilen *tum_yollar* arasından en küçük ağırlığa sahip olan yol döndürülür ve en kısa yol yine soldaki çıktı ekranında ağırlığı ile beraber yazdırılır. Bu en kısa yol, shortest_path_1 değişkeninde saklanır ve daha sonra *grafi_ciz* fonksiyonu çağrılır ve ilgili yol, graf görselinde üzerinden geçilen düğümlerin etrafı ve kenarlar kırmızı renkle gösterilir.

2. İSTER: Kullanıcı tarafından ORCID değeri girilen yazarın ve onun iş birliği yaptığı yazarlar düğüm ağırlıklarına yani yazdıkları makale sayılarına göre sıralanmaktadır. Daha önce yapılan işlemlerden kalan değişikliklerin silinmesi için *grafi_sifirla* fonksiyonu ile grafik temizlenir. *QInputDialog.getTex*t ile kullanıcıdan iki id alınır ve *ok_a, ok_b* değerleri id yazıldıktan sonra butona tıklanıp tıklanmadığını kontrol eder. Girilen id değerine sahip yazarlar bulunursa *agirlikli_kuyruk_olustur* fonksiyonu çağırılır. Bu fonksiyon bir yazarı parametre olarak alır ve kuyruğa yeni yazar ekleyerek bunları düğüm ağırlıklarına göre sıralar. Kullanıcıya her seferinde eklenen yazarı ve yazar kuyruğunun yeni sırasını adım adım göstermek için *QTimer* modülü ile bir zamanlayıcı ayarlanmış ve her eleman eklendikten sonra kuyruğun yeni durunu göstermek için bir saniye beklenmiştir. Bu fonksiyonun içinde tanımlı olan *komsulari_ekle* fonksiyonu ile de yazarların tüm iş birliği yaptığı yazarlar gezilir. Daha sonra *append()* ile kuyruğa eklenirler. Bu kuyruk *sort()* ile sıralanır ve en büyük ağırlığa sahip düğüm yani en çok makale yazmış yazar en başa konumlandırılır. Tüm komşular kuyruğa eklendiyse zamanlayıcı durdurulur.

3. İSTER: İlk isterde oluşturulan en kısa yoldaki kuyruktan bir Binary Search Tree oluşturulmakta ve kullanıcının girdiği id'ye sahip yazar bu ağaçtan silinmektedir. *grafi_sifirla(reset_shortest_path = FALSE)* ile grafı sıfırlar ama en kısa yol sıfırlanmaz, korunur. *shortest_path_1* değişkeninin daha önce hesaplanıp hesaplanmadığı kontrol edilir ve onun üzerinde işlem yapılır. Eğer en kısa yol bulunmamışsa kullanıcıya en kısa yol bulunması gerektiği uyarısı verilir. Bu yoldaki yazarlar *path_bilgisi* içinde oklarla art arda eklenir ve *output_area.append* ile soldaki çıktı ekranına yazdırılır. Daha sonra bu yoldaki yazarları ve yazdıkları makale sayılarını içeren bir kuyruk oluşturulur ve bu kuyruk, Graf sınıfında tanımlanan ve işleyişi "Sınıfların Oluşturulması" bölümünde anlatılan *kuyruktan_bst_olustur* fonksiyona parametre olarak gönderilir. Buradan oluşturulan ağacın kökü döner ve *bst_root* içinde saklanır. Daha sonra *bst_root, en_kisa_yol_agacini_ciz* fonksiyonuna parametre olarak gönderilir ve ağaç görselleştirilir. Fonksiyon içindeki *dugum_ekle* fonksiyonu, BST'deki her bir düğüm için pozisyonları hesaplayan ve sahneye çizen bir fonksiyondur. Bir değer döndürmez, sadece görselleştirmeye yarar. Her düğüm için kendisini tekrar çağırarak ağacın tüm düğümlerini yerleştirir yani bu fonksiyon recursive olarak çalışır. Düğümler çizilirken yine graf düğümü çizimindeki kurallar takip edilir ve yazarların yazdığı makale sayısına göre büyüklükleri ve renkleri ayarlanır. Aynı graf görselinde olduğu gibi burada da düğümlere karşılık gelen *TiklanabilirElips* nesnesi oluşturulur ve düğüm koordinatlarına göre çizilir. Daha sonra düğümler arasındaki bağlantılar için de düğümlerin sağ ve sol çocukları arasında çizgiler çizilir. Ağaçtan yazar silmek için *QInputDialog.getText* ile kullanıcıdan bir ORCID numarası alınır ve bu yazar bstden_dugum_sil fonksiyonuna gönderilir. Eğer silinecek yazarın makale sayısı, mevcut düğümün makale sayısından küçükse, sol alt ağaca

yönlendirilir. Eğer silinecek yazarın makale sayısı, mevcut düğümün makale sayısından büyükse, sağ alt ağaca tam tersi sağ ağaca yönlendirilir. ORCID'ler karşılaştırılarak silinecek düğüm bulunur. Silinecek düğümün iki çocuğu yoksa, yalnızca mevcut olan alt ağaç döndürülür. Ancak, iki çocuğu varsa, sağ alt ağaçtaki en küçük düğüm bulunur ve bu düğüm silinen düğümün yerine yerleştirilir. Böylece BST kurallarına uyularak ağacın yapısı korunmuş olur.

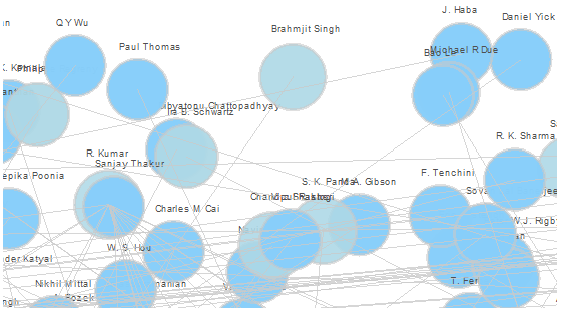
4. İSTER: Kullanıcıdan id'si girilen yazarın iş birliği yaptığı yazalar ve onların da iş birliği yaptığı yazarların yolları belirlenecektir. Başlangıç düğümünden gidilebilecek tüm düğümlere giden en kısa yollar ağırlıklarına göre hesaplanacak ve listelenecektir. Yine *QInputDialog.getText* kullanılarak kullanıcıdan bir ORCID değeri alınır ve *ok_a* True ise değer alınır ve graf düğümlerinde aranarak yazar bulunur. Seçilen yazarla doğrudan veya dolaylı olarak bağlantısı olan tüm yazarları bulmak için başlangıçta, sadece seçilen yazarın eklendiği *to_visit* adlı bir liste ve bağlantılı yazarları saklayan bir *baglantili_tum_yazarlar* adlı bir set oluşturulur. Liste boşalana kadar döngünün her adımında bir yazar işlenir ve bu yazarın doğrudan bağlantılı olduğu yazarlar kontrol edilir. Eğer bağlantılı bir yazar daha önce işlenmemişse listeye eklenir. Bu işlem bağlantısı olan tüm yazarlar bulunana kadar devam eder. Set kullanımı sayesinde aynı yazarın birden fazla kez işlenmesi önlenir ve tüm bağlantılar tamamlandığında seçilen yazarla bir şekilde ilişkili olan tüm yazarlar kaydedilmiş olur. "Sınıfların Oluşturulması" kısmında işlevi açıklanan *tum_yollar_bul* fonksiyonu çağırılarak ana yazar ve diğer yazarlar arasında tüm yollar hesaplanır. Daha sonra tüm bu düğümler için *en_kisa_yollar_hesapla* fonksiyonu ile en kısa yollar hesaplanır ve *en_kisa_yollar* değişkenine kaydedilir. Ana yazardan ilişkili yollara gidilen yolların bilgileri ve bu düğümlere giden en kısa yolların bilgisi soldaki çıktı ekranına yazdırılır. *Graf4()* fonksiyonu çağırılarak otak segmentteki graf görseli kalkar ve yerine ana yazar ve onun gidebildiği tüm düğümler *TiklanabilirElips* nesnesi olarak ekranda teker teker belirirler, analizin daha kolay olması için yazarların üzerinde isimleri de daha büyük punto ile gösterilir. Aynı zamanda kenarların da üzerine tıklanarak kenar ağırlıklarının doğrulu kontrol edilir.

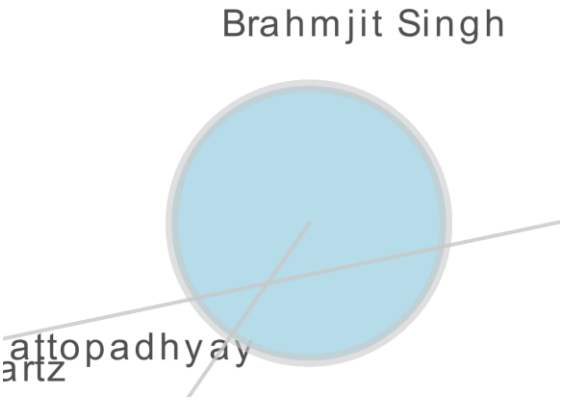
5. İSTER: Kullanıcının girdiği id'ye sahip yazarın iş birliği yaptığı yazar sayısı gösterilecektir. Yine işlemler başlamadan önce *self.grafi_sifirla()* komutu ile graf görseli temizlenir. Yine *QInputDialog.getText* ile kullanıcıdan bir ORCID değeri alınır, ok_a True ise kullanıcı geçerli bir değer girmiştir. Yine id tüm düğümler içinde aranır ve yazar bulunur. Daha sonra daha önce işleyişi açıklanan Graf sınıfının bir fonksiyonu olan ve işleyişi "Sınıfların Oluşturulması" bölümünde anlatılan *ortak_yazar_sayisi_hesapla* fonksiyonuna bu yazar parametre olarak gönderilir ve yazarın toplam iş birliği sayısı bulunur. Graf üzerinde bu yazar ve onun iş birliği yaptığı yazar düğümleri kırmızı renkte gösterilmiştir. Bunu yapabilmek için de *self.scene.items()* içinde tüm graf nesneleri taranır ve nesne *TıklanabilirElips* türündeyse ve girilen yazar nesnesiyse, etrafına kırmızı kenarlık yapılır. Yazarın iş birliği yaptığı yazarların boyanması için ise yazarın *connections* listesindeki yazarlar döngü ile taranır ve bu yazarlar *TıklanabilirElips* olarak işaretlenir. İşaretleme yine kırmızı kenarlık ile yapılır. Bu işlem PyQt5'in *QPen* modülüne *QColor* değeri ve kalınlığı girilerek yapılır. Daha sonra ana yazar ve iş birliği yaptığı yazarlar arasında çizgi çizilir.

6. İSTER: Veri tabanındaki yazarlar içinden en çok iş birliği yapmış yazar gösterilecektir. Butona tıklandığında yine *grafi_temizle* fonksiyonu ile graf üzerindeki işaretlemeler temizlenir. Daha önce "Sınıfların Oluşturulması" bölümünde işleyişi açıklanan ve Graf sınıfının bir fonksiyonu olan *en_cok_isbirligi_yapan_yazari_bul* fonksiyonu çağırılır ve dönen yazar *en_cok_isbirligi_yapan_yazar* değişkenine, toplam bağlantı sayısı ise *toplam_isbirligi* değişkenine atanır. En çok iş birliği yapan yazar ve sayısı soldaki çıktı ekranına *output_area.append* ile yazdırılır. Bu yazarı graf görselinde göstermek için ise tüm düğümler taranır. Eğer düğüm *TiklanabilirElips* türündeyse ve *en_cok_isbirligi_yapan_yazar* nesnesine eşitse kırmızı kenarlıkla belirtilir.

7. İSTER: Kullanıcının girdiği id'ye sahip yazardan başlayarak, üzerinden geçilen düğüm sayısına göre

gidilebilecek en uzun yol belirlenir. Yine, önceki işlemlerden kalan graf üzerindeki değişiklikler *grafi_sifirla()* fonksiyonu ile temizlenir. *QInputDialog.getText* ile kullanıcıdan bir yazar ORCID değeri alınır. *ok_a* değişkeni True ise kullanıcı değeri girmiştir. Girilen id değerine sahip yazar graf düğümleri arasında aranır. Yazar bulunursa *yazar_a* değişkenine atanır. İşleyişi daha önce açıklanan ve Graf sınıfının bir fonksiyonu olan ve işleyişi "Sınıfların Oluşturulması" bölümünde açıklanan *en_uzun_yol_bul* fonksiyonuna *yazar_a* yazarının orcid değeri parametre olarak gönderilir ve dönen liste, *longest_path* değişkenine kaydedilir. Daha sonra bu yol, soldaki çıktı ekranında *output_area.append* ile gösterilir ve *grafi_ciz()* fonksiyonu çağrılır. Bu fonksiyonda kullanılacak olan *longest_path* None olmadığından if blokuna girilir ve yol üzerindeki düğümler ve kenarlar kırmızı kenarlıklarla gösterilir.

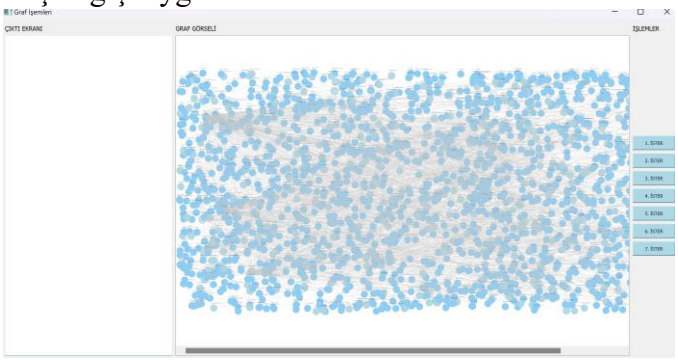




IX. Deneysel Sonuçlar

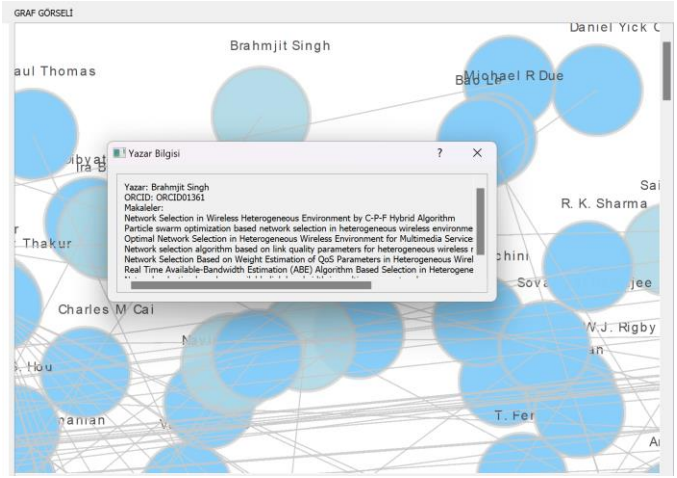
Veri setinden girilen çeşitli ORCID değerleri ile isterlerin çıktıları kontrol edilmiş ve çıktılar graf görseli üzerinde teyit edilmiştir. Büyük veri setine geçmeden önce, excel dosyasındaki formatta oluşturulan küçük boyutlu bir dosya üzerinde denemeler yapılmış ve fonksiyonların doğruluğu ispatlanmıştır.

Başlangıç Uygulama Görseli:



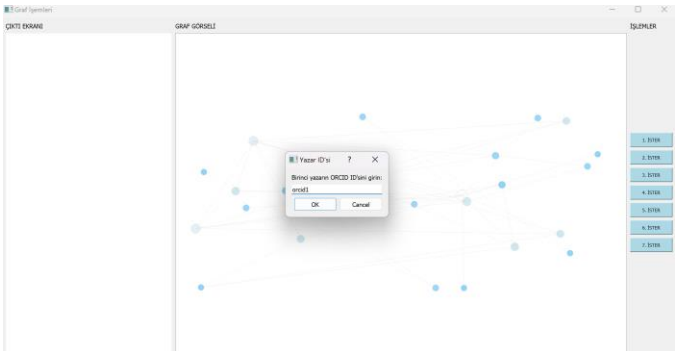
Zoom-in zoom-out özelliği:

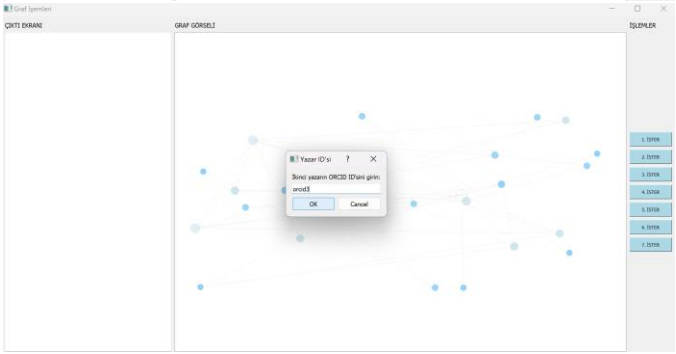
Tıklanabilir düğümler:



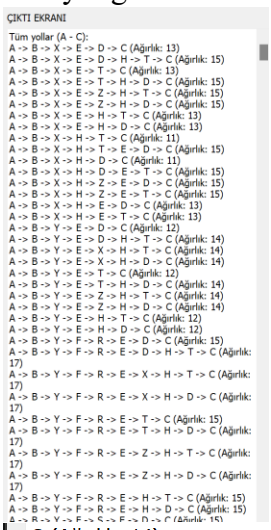
1. İster:

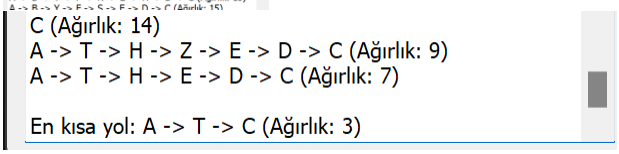
Bu isteri denemek için, aynı formatla oluşturulan, 23 eşsiz yazarın olduğu dosya kullanılmıştır ve o dosyada gözlemlenen sonuçlar program üzerinde denenmiştir. Önce kullanıcıdan iki adet ORCID değeri alınır.



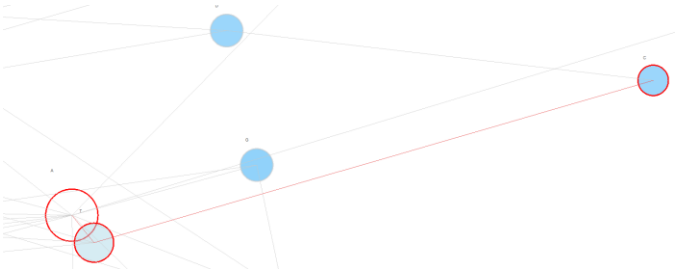


Daha sonra sol taraftaki çıktı ekranında iki yazar arasındaki tüm düğümler listelenir ve en sonda da en kısa yol gösterilir.



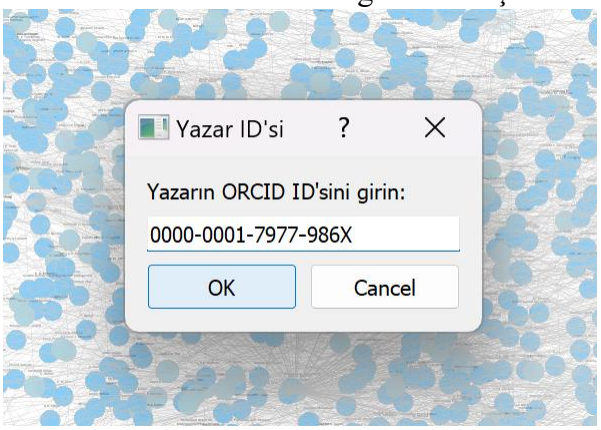


Bu en kısa yol graf üzerinde düğümlerin kenarları kenarlar kırmızı yapılarak gösterilir.



2. İster:

Kullanıcıdan bir ORCID değeri alınmıştır.



Girilen yazarın tüm bağlantıları teker teker eklenerek soldaki çıktı ekranında her adım sıralı olarak listelenmiştir.

En sonunda da tüm bağlantıların dahil olduğu listed sıralı olarak solda gösterilmiştir.

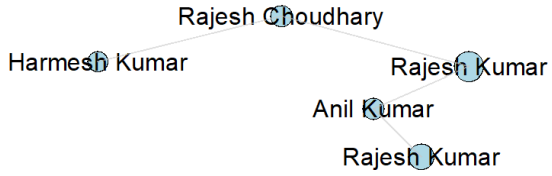
```
Yazar eklendi: Deepika Poonia, Makale Sayısı: 1
Güncel Kuyruk:
  1. Rajesh Kumar (Makale Sayısı: 19)
  2. Priyanka Yogi (Makale Sayısı: 15)
  3. Suryakant Mishra (Makale Sayısı: 13)
  4. Shailendra K. Saxena (Makale Sayısı: 8)
  5. Anil Kumar (Makale Sayısı: 8)
  6. Pankaj R. Sagdeo (Makale Sayısı: 7)
  7. Anjali Chaudhary (Makale Sayısı: 6)
  8. Devesh K. Pathak (Makale Sayısı: 6)
  9. Swarup Roy (Makale Sayısı: 5)
  10. P. R. Sagdeo (Makale Sayısı: 5)
  11. Manushree Tanwar (Makale Sayısı: 3)
  12. Vikash Mishra (Makale Sayısı: 3)
  13. Vinayak Mishra (Makale Sayısı: 3)
  14. Vivek Kumar (Makale Sayısı: 3)
  15. M. Kamal Warshi (Makale Sayısı: 2)
  16. P.R. Sagdeo (Makale Sayısı: 2)
  17. Simran Lambora (Makale Sayısı: 2)
  18. Aanchal Sati (Makale Sayısı: 2)
  19. Archna Sagdeo (Makale Sayısı: 2)
  20. M Kamal Warshi (Makale Sayısı: 1)
  21. Ambadas B Phatangare (Makale Sayısı: 1)
  22. Sanjay Dhole (Makale Sayısı: 1)
  23. V N Bhoraskar (Makale Sayısı: 1)
  24. Harnath Ghosh (Makale Sayısı: 1)
  25. P R Sagdeo (Makale Sayısı: 1)
  26. Deepak Gupta (Makale Sayısı: 1)
  27. Shailendra Saxena (Makale Sayısı: 1)
  28. Pankaj R Sagdeo (Makale Sayısı: 1)
  29. Alexander S. Krylov (Makale Sayısı: 1)
  30. Puspen Mondal (Makale Sayısı: 1)
  31. Bipin Joshi (Makale Sayısı: 1)
  32. Deepika Poonia (Makale Sayısı: 1)
Tüm yazarlar sıralandı.
```

Graf üzerinde ise, önce id'si girilen yazarın etrafı kırmızı çizgiyle kaplanır ve sırayla eklenen tüm bağlantılar kırmızı renkte gösterilir.

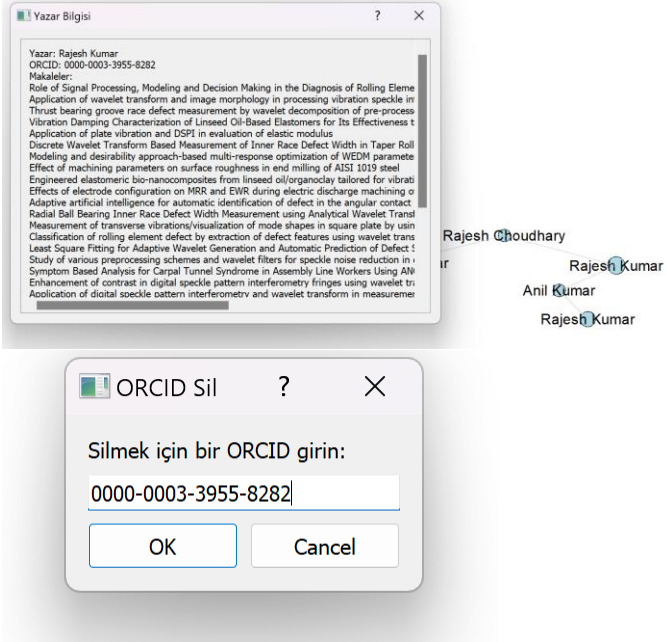
3. İster: İlk isterde girilen yazarları alır ve o kuyruk üzerinden bir BST oluşturur.

ÇIKTI EKRANI

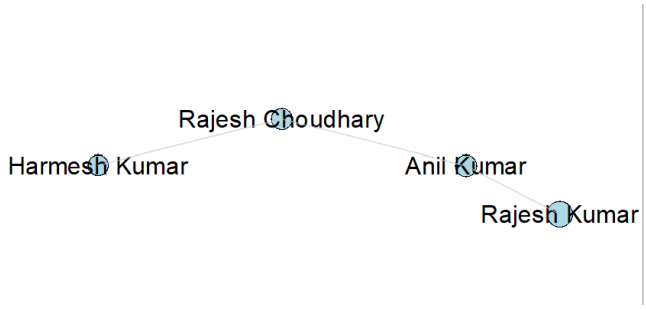
```
En kısa yol: Rajesh Choudhary (ORCID:
0000-0002-8955-0062) -> Harmesh Kumar (ORCID:
ORCID01177) -> Rajesh Kumar (ORCID:
0000-0003-3955-8282) -> Anil Kumar (ORCID: ORCID01575)
-> Rajesh Kumar (ORCID: 0000-0001-7977-986X)
```



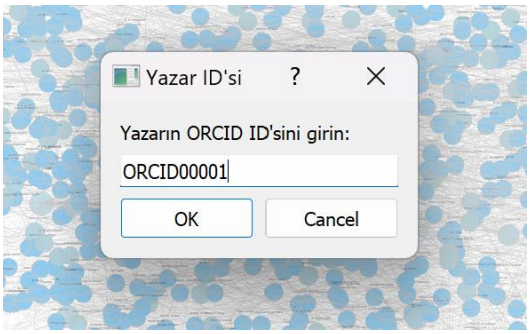
Daha sonra silinecek düğümün ORCID değeri kullanıcıdan alınır. Yazılacak ORCID değeri ağaç üstünde düğümlere tıklanarak incelenebilmektedir.



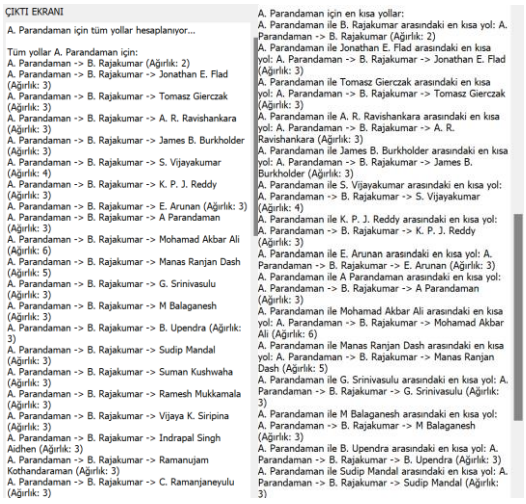
Daha sonra seçilmiş yazar silinerek ağaç tekrar gösterilir.



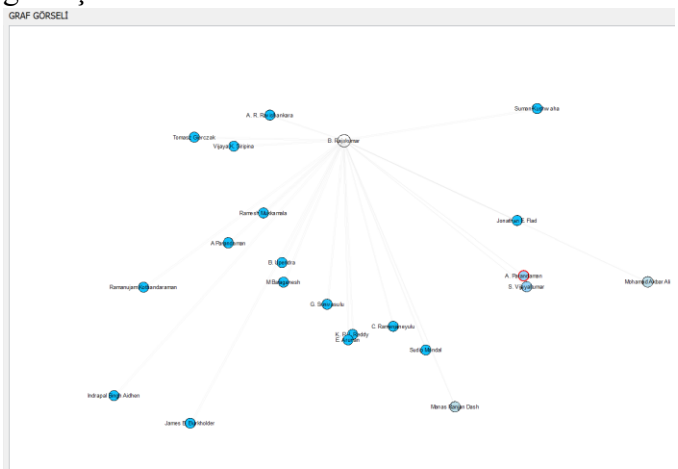
4. İster:
Kullanıcıdan bir yazarın ORCID değeri alınır.



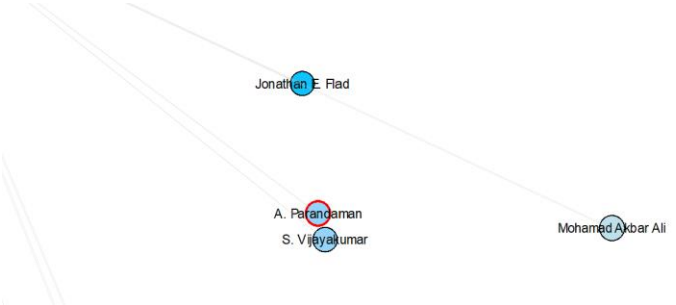
Girilen bu yazarın dolaylı ya da doğrudan tüm bağlantıları çıktı ekranında listelenir. Daha sonra da bu yollardan, her düğüme olan en kısa yollar listelenir.



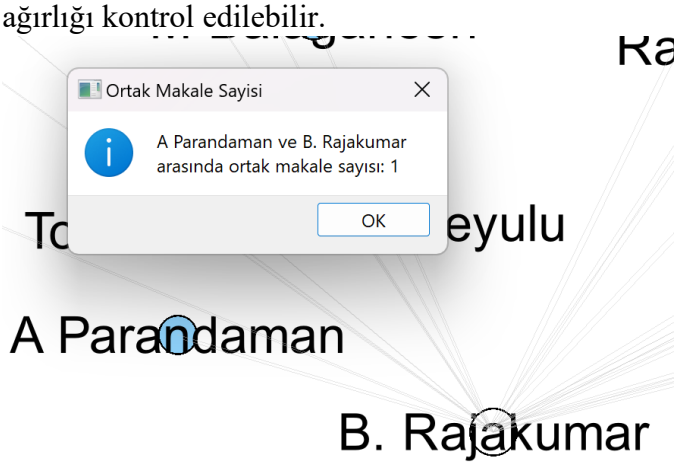
Bu esnada orta segmentteki graf görseli yerini bu yazar ve bağlantılar ağını gösteren bir görsel gelmiştir.



Girilen yazarın etrafı kırmızı kenarlıkla sarılır.

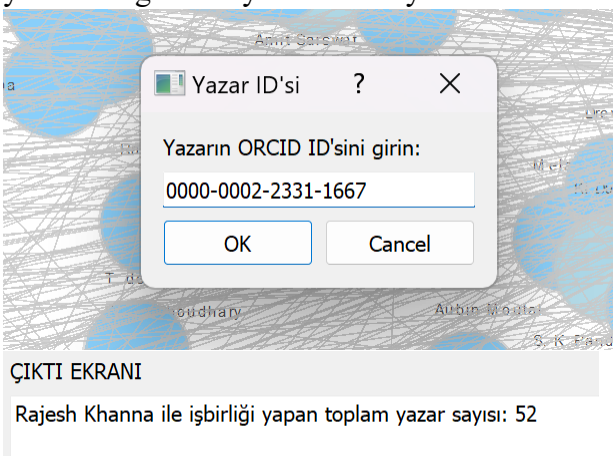


Kenarlar tıklanabilir olmuştur, iki yazar arasındaki ortak makale sayısı görülebilir ve bu sayede yolların ağırlığı kontrol edilebilir.

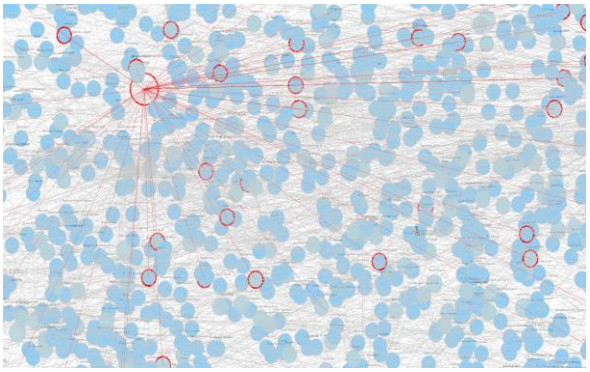


5. İster:

Kullanıcıdan bir ID girilmesi istenir ve girilen yazarın bağlantı sayısı ekranda yazdırılır.



Aynı zamanda graf üzerinde ilgili yazar ve bağlantıları kırmızı renkle gösterilir.

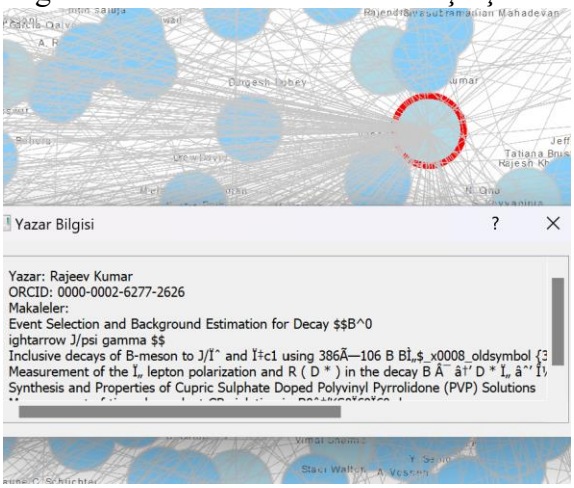


6. İster:
En çok iş birliği yapan yazar çıktı ekranında gösterilir.

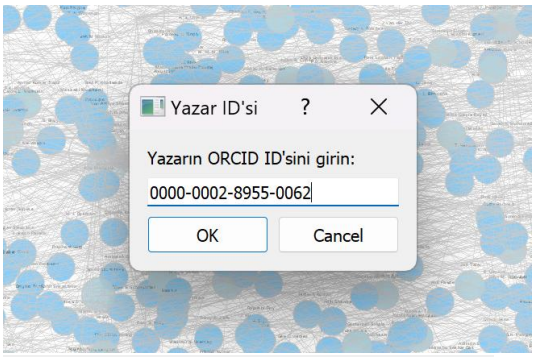
ÇIKTI EKRANI

En cok işbirliği yapan yazar: Rajeev Kumar, Toplam İşbirliği: 227

Aynı zamanda graf üzerinde ilgili yazarın düğümünün etrafı kırmızı renk ile çerçevelenir.

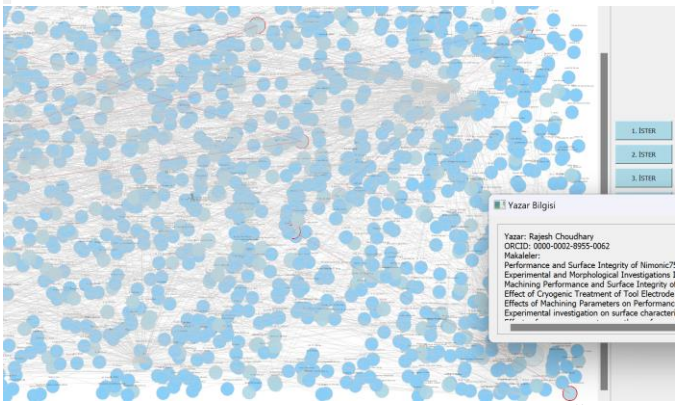


7. İster:
Kullanıcıdan bir ORCID girilmesi istenmiş ve sonuç olarak bu yazardan gidilebilecek geçilen düğüm sayısı baz alınarak en uzun yol soldaki çıktı ekranında, aynı zamanda da graf üzerinde bir yol olarak kırmızı renkle gösterilmiştir.



ÇIKTI EKRANI

En uzun yol: Rajesh Choudhary -> Harmesh Kumar -> Rajesh Kumar -> Manoj Kumar -> Rajesh Kumar -> Amit Kumar -> Rajeev Singh -> Rajiv Kumar -> Rajesh Khanna -> Jaswinder Kaur -> Rajesh Kumar -> Peter Bienstman (Düğüm Sayısı: 12)



Tüm isterler PROLAB 3 - GÜNCEL DATASET.xlsx dosyasındaki çeşitli ORCID numaralarıyla denenerek fonksiyonların doğruluğu teyit edilmiştir.

X. Kaba Kod

Kod içerisinde bulunan sınıflar ve fonksiyonların kaba kodu aşağıdadır.

```
 Başla

Sınıf Yazar:
   Özellikler:
      orcid: Yazarın ORCID kimliği
      isim: Yazarın adı
   makaleler: Yazarın yazdığı makalelerin
listesi (boş liste)
  bağlantılar: Yazarın diğer yazarlarla olan
bağlantılar (boş liste)

 Metotlar:
     repr(): Yazarın temsilini döndürür.
Örneğin: "Yazar(İsim, ORCID)"
```

Sınıf Makale:
   Özellikler:
   doi: Makalenin DOI numarası
   başlık: Makalenin başlığı
 yazarlar: Makaleye katkıda bulunan yazarların listesi (boş liste)

  Metotlar:
      repr():
   Makalenin temsilini döndürür. Örneğin: "Makale(Başlık, DOI)"

Sınıf Graf:
   Özellikler:
    düğümler: Grafın düğümlerini (yazarları) tutan liste (boş liste)

    Metotlar:
   dugum_ekle(yazar):
          Eğer yazar grafta yoksa, yazarı düğümler listesine ekler.

   kenar_ekle(yazar1, yazar2, ortak_makale_sayisi):
       İki yazar arasında bir kenar ekler. Eğer yazar1'in bağlantılarında yazar2 yoksa, yazar2'yi ekler ve ortak makale sayısını kaydeder.

 tum_yollar_bul(yazar_a, yazar_b, yol, toplam_agirlik):
      İki yazar arasındaki tüm yolları bulur. Eğer yazar_a ve yazar_b aynıysa, mevcut yolu ve toplam ağırlığı döndürür. Aksi takdirde, komşular üzerinden rekürsif olarak yolları arar.

      en_kisa_yol_bul(tum_yollar):
         Verilen yollar arasından en kısa olanını bulur ve döndürür.

      en_kisa_yollar_hesapla(yazar):
         Belirtilen yazar için diğer yazarlara olan en kısa yolları hesaplar ve sonuçları bir sözlükte döndürür.

      kuyruktan_bst_olustur(kuyruk):
          Verilen kuyruktan bir ikili arama ağacı (BST) oluşturur. Her yazarın makale sayısına göre düğümleri yerleştirir.

      ortak_yazar_sayisi_hesapla(yazar):
      Belirtilen yazarın kaç tane ortak yazarı olduğunu döndürür.

      en_cok_isbirligi_yapan_yazari_bul():
        Graf içindeki en çok işbirliği yapan yazarı bulur ve toplam işbirliği sayısını döndürür.

      en_uzun_yol_bul(yazar_id):
          Belirtilen yazar için en uzun yolu bulur ve döndürür.

Ana Program:
 dosya = "PROLAB 3 - GÜNCEL DATASET.xlsx"
     veriseti = Excel dosyasını oku

 yazarlar = boş sözlük
 makaleler = boş sözlük
 yazar_ismi_haritası = boş sözlük
 orcid_sayaci = 1

 graf = Graf()

 veriseti üzerinden her satır için:
     orcid, author_name al
     Eğer yazar yoksa:
        yeni_yazar oluştur ve ekle
     Aksi takdirde:
        mevcut yazarı güncelle

  veriseti üzerinden her satır için:
      orcid, doi, paper_title, author_position al
     coauthors listesini al
     merkez_yazarı bul
     makaleyi ekle
     yazarın makalelerine ekle
     ortak yazarları ekle

 graf.düğüm_ekle(yazar) ile tüm yazarları graf'a ekle

Sınıf TiklanabilirElips:
   Özellikler:
     x: Elipsin x koordinatı
     y: Elipsin y koordinatı
     radius: Elipsin yarıçapı
     author: Elipsin temsil ettiği yazar

  Metotlar:
     mousePressEvent(event):
       Elips tıklandığında yazar bilgilerini gösteren bir diyalog açar.

Sınıf TiklanabilirCizgi:
  Özellikler:
     start_author: Başlangıç yazarını temsil eder
     end_author: Bitiş yazarını temsil eder
   ortak_makale_sayisi: İki yazar arasındaki ortak makale sayısı

   Metotlar:
      mousePressEvent(event):
        Cizgi tıklandığında iki yazar arasındaki ortak makale sayısını gösteren bir mesaj kutusu açar.

```
Sınıf GrafArayuzu:
      Özellikler:
         pencere başlığı, boyut, layout,
butonlar, vb.

      Metotlar:
         grafi_sifirla():
         Grafi sıfırlar ve gerekli
değişkenleri günceller.

         bst_ciz(root):
                   İkili arama ağacını in-
order olarak çizer.

          al_en_kucuk(node):
               Verilen düğümden en küçük
düğümü bulur.

          bstden_dugum_sil(root, author):
                  İkili ağaçtan belirtilen
yazarı siler.

          grafi_ciz():
          Grafı çizer ve yazarların
konumlarını belirler.

          en_kisa_yol_agacini_ciz(bst_root):
                    En kısa yol ağacını çizer.
          yazar_bilgisi_goster(author):
                   Belirtilen yazarın
bilgilerini gösteren bir mesaj kutusu açar.

          wheelEvent(event):
                   Fare tekerleği ile zoom
işlemi yapar.

          agirlikli_kuyruk_olustur(author):
           Belirtilen yazar ve işbirliği
yaptığı yazarları bir kuyrukta toplar.

        graf4(authors, main_author):
            Belirtilen yazarlar için grafı
çizer.

        buton_tiklandi():
          Tıklanan butona göre gerekli
işlemleri yapar.

Eğer main ise:
   Uygulamayı başlat
   Pencereyi göster
   Uygulamayı çalıştır

Bitir
```

XI. SONUÇ

Bu yazarlar ve iş birliği analizi projesi, bir veri tabanından alınan verileri saklama biçimine göre analiz edip ayrıştıran ve yazarlarla iş birliklerini temel alan bir graf yapısı oluşturan bir Python programıdır. Bu projenin ana bileşenleri veri dosyasının ayrıştırılması, ayrıştırılan verilerden manuel bir graf yapısı oluşturulması ve graf üzerinde yapılan işlemlerin kullanıcıya sunulmasıdır. Oluşturulan bu graf yapısı interaktif bir uygulama içinde kullanıcıya görsel olarak sunulmuş ve yapılan işlemlerin sonuçları graf üzerinde gözlemlenmiştir.
Gerçekleştirilen işlemlerin en kısa ve en uzun yolun bulunması, belirli kriterlere göre sıralama, çıkarma, ekleme, arama, ikili arama ağacı oluşturma gibi veri yapılarının temel işlemleridir ve bu sayede bilgisayar bilimlerinin önemli bir bölümü olan veri saklama ve analizi için temel oluşturan bu çeşitli veri yapıları işlemleri uygulama deneyimi edinilmiştir.
Sonuç olarak küçük çaplı bir veri setini ayrıştırma ve modelleme işlemlerini manuel olarak yapan öğrenciler için oldukça öğretici bir proje olmuştur.

KAYNAKLAR

[1] ”Graf (Şekil, Graph)”. BilgisayarKavramlari. 1 Ağustos 2008. https://bilgisayarkavramlari.com/2008/08/01/graf-sekil-graph/

[2] “Graph nedir?” . Serhanbaymaz. 15 Aralık 2022. https://medium.com/@serhanbaymaz/graph-nedir-4046bad39529

[3] “Graph Implementation in Python”. Shivam Bahatele. 20 Nisan 2023. https://experiencestack.co/graph-implementation-in-python-916fc3b6a8a

[4] “Applications of Graph Data Structure”. Geekforgeeks. 24 Şubat 2023. https://www.geeksforgeeks.org/applications-of-graph-data-structure/

[5] “What is an ORCID id and how do I use it?”. ORCID. Erişim Tarihi: 27 Aralık 2024. https://support.orcid.org/hc/en-us/articles/360006897334-What-is-an-ORCID-iD-and-how-do-I-use-it

[6] "Pandas Tutorial". w3schools. Erişim Tarihi: 27 Aralık 2024. https://www.w3schools.com/python/pandas/default.asp
[7] "pandas - Python Data Analysis Library". pandas. Erişim Tarihi: 27 Aralık 2024. https://pandas.pydata.org/
[8] "pandas.read_excel" . Pandas. Erişim Tarihi: 27 Aralık 2024. https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html
[9] "İkili Arama Ağacı (Binary Search Tree)". Şadi Evren Şeker. 7 Mayıs 2008. https://bilgisayarkavramlari.com/2008/05/07/ikili-arama-agaci-binary-search-tree/
[10] "Writing a Binary Search Tree in Python with Examples" . Layne Wagner. 1 Ekim 2022. https://blog.boot.dev/computer-science/binary-search-tree-in-python/
[11] "PyQt5 5.15.11". PyQt5 . PyPI. 19 Temmuz 2024. https://pypi.org/project/PyQt5/#:~:text=PyQt5%20is%20a%20comprehensive%20set,platforms%20including%20iOS%20and%20Android.
[12] "Python PyQt5 setup a basic GUI Application!". Bro Code. 23 Temmuz 2024. https://www.youtube.com/watch?v=UvqmV3PGiXM
[13] "Generate a graph using Dictionary in Python". geeksforgeeks. 13 Eylül 2023. https://www.geeksforgeeks.org/generate-graph-using-dictionary-python/
[14] "Graph Introduction – Data Structures & Algorithms Tutorials in Python". codebasics. 22 Ağustos 2020. https://www.youtube.com/watch?v=j0IYCyBdzfA&list=PLpOCemruBVVvle7KRKaozEJ12fbAS6-mM&index=2